

LeetCode Deep Dive

From First Instinct to Optimal Algorithm: Game DP and State-Space BFS in Practice
Week 13 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

Outline

Why walk through LeetCode problems?

- This week's LEETCODE.md list gives you eight Week 13 problems to *practice on your own*. This deck instead **narrates the thinking process** for two of them, end to end, so you can reuse the process on the rest.
- We pick one **Medium** (*Stone Game*) and one **Hard** (*Word Ladder*) – one is a two-player **game** solved by memoized minimax-style DP, the other is a huge implicit **state space** tamed by BFS with pruning.
- Both are exactly this week's two big ideas in miniature: a game whose exponential-looking tree collapses to a polynomial number of *distinct* states, and a search space too large to enumerate, made tractable by exploring it breadth-first with a visited set instead of blindly.

The four-step process we will repeat twice

- ① **First instinct.** Write down the algorithm that follows most directly from re-reading the problem statement – usually “simulate every choice both players could make” or “search every path.”
- ② **Measure why it hurts.** Compute the actual complexity, plug in the problem’s constraints, and see the number blow up.
- ③ **Find the structural insight.** Ask: *how many genuinely distinct states are there, really?* Almost always far fewer than the number of *paths* to reach them.
- ④ **Implement the optimal algorithm,** trace it by hand on a small example, then look for the *programming-level* (not algorithmic) tricks that make the implementation itself correct and fast.

This mirrors the course

Steps 1–3 are this week’s “exponential tree $\rightarrow$ polynomially many *states*” story (minimax $\rightarrow$ memoized game DP, bounded search trees); step 4 turns a correct algorithm into working code.

Problem statement

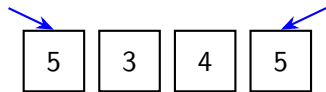
LeetCode 877 – Stone Game ([Medium](#))

An **even** number of piles of stones are arranged in a row; pile i has `piles[i]` stones (the total is odd, so no ties are possible). Alex and Lee alternate turns, Alex first; each turn, a player removes the entire pile from **either end** of the remaining row and adds its stones to their own score. Both play **optimally** to maximize their own final score. Return `true` if Alex wins.

- Constraints (LeetCode): $2 \leq \text{piles.length} \leq 500$ (even), $1 \leq \text{piles}[i] \leq 500$.
- This is exactly this week's **minimax** game tree: at each node, the player to move picks the child (take-left or take-right) that is best *for them*, and the two players alternate the max/min role.

Worked example

piles = [5, 3, 4, 5]



each turn: take the leftmost or rightmost remaining pile

- Alex moves first, then Lee, alternating, until all 4 piles are gone.
- Both play optimally for *themselves* – Alex is not trying to minimize Lee's score, only to maximize his own (the classic subtlety of general-sum-looking but actually zero-sum-difference games).
- **Answer:** true – with *both* playing optimally, Alex wins 9 stones to Lee's 8 (we derive this score exactly, not by guessing, on the trace slide below).

First instinct: simulate the whole game tree

Most direct reading of the problem

“Both play optimally” $\Rightarrow$ recursively try both choices (take-left, take-right) for whoever's turn it is, and let each player pick whichever leads to the better outcome *for them*.

```
1: function PLAY(piles, turn)
2:   if piles is empty then return (0, 0)                                ▷ (Alex total, Lee total)
3:   end if
4:   left  $\leftarrow$  PLAY(piles without first pile,  $\neg$ turn)                ▷ add first pile to turn's side
5:   right  $\leftarrow$  PLAY(piles without last pile,  $\neg$ turn)                ▷ add last pile to turn's side
6:   return whichever of left, right gives turn the larger score
7: end function
```

Why this is bad

- Each call branches into 2 recursive calls (take-left or take-right), and the recursion is n turns deep $\Rightarrow O(2^n)$ leaves.
- Plug in the constraint: $n = 500 \Rightarrow 2^{500} \approx 3 \times 10^{150}$ – vastly more than the number of atoms in the observable universe ($\sim 10^{80}$). This is not “slow,” it is **uncomputable** in the lifetime of the universe.

The smell to notice

The recursion tree has 2^n leaves, but look at what each call actually depends on: only *which contiguous slice of piles remains*. Many different take-left/take-right *sequences* land on the exact same remaining slice – that repetition is the wasted work.

The structural insight

- After any sequence of moves, the remaining piles are always a **contiguous window** $\text{piles}[i..j]$ – there are only $\binom{n+1}{2} = O(n^2)$ such windows, versus 2^n move sequences.
- Reformulate with a **score-difference** trick: let $dp[i][j]$ be the largest possible (*current player's score* – *opponent's score*) achievable by whoever moves first on window $[i, j]$. Then

$$dp[i][j] = \max(\text{piles}[i] - dp[i+1][j], \text{piles}[j] - dp[i][j-1]),$$

with base case $dp[i][i] = \text{piles}[i]$.

- Alex wins iff $dp[0][n-1] > 0$ – because $dp[0][n-1]$ is exactly (Alex's total) – (Lee's total) under optimal play by both.

Optimal algorithm: pseudocode

```
1: function STONEGAME(piles)
2:    $n \leftarrow \text{len}(\text{piles})$ 
3:    $dp[i][i] \leftarrow \text{piles}[i]$  for all  $i$ 
4:   for  $len \leftarrow 2$  to  $n$  do
5:     for  $i \leftarrow 0$  to  $n - len$  do
6:        $j \leftarrow i + len - 1$ 
7:        $dp[i][j] \leftarrow \max(\text{piles}[i] - dp[i+1][j], \text{piles}[j] - dp[i][j-1])$ 
8:     end for
9:   end for
10:  return  $dp[0][n-1] > 0$ 
11: end function
```

▷ windows of length 1
▷ grow the window size

$O(n^2)$ time, $O(n^2)$ space – versus the $O(2^n)$ naive tree.

Trace on the worked example

piles = [5, 3, 4, 5]; fill $dp[i][j]$ by increasing window length.

window	length 1	length 2	length 3 / 4
[0, 0], [1, 1], [2, 2], [3, 3]	5, 3, 4, 5		
[0, 1]		$\max(5-3, 3-5) = 2$	
[1, 2]		$\max(3-4, 4-3) = 1$	
[2, 3]		$\max(4-5, 5-4) = 1$	
[0, 2]			$\max(5-dp[1, 2]=1, 4-dp[0, 1]=2) = \max(4, 2) = 4$
[1, 3]			$\max(3-dp[2, 3]=1, 5-dp[1, 2]=1) = \max(2, 4) = 4$
[0, 3]			$\max(5-dp[1, 3]=4, 5-dp[0, 2]=4) = \max(1, 1) = 1$

$dp[0][3] = 1 > 0 \Rightarrow \mathbf{true}$ – matches LeetCode's expected output. Unwinding *which* choice achieved each max gives one concrete optimal line: Alex takes pile 0 (5), Lee takes pile 3 (5), Alex takes pile 2 (4), Lee takes pile 1 (3) – Alex 9, Lee 8, difference 1, totalling 17. ✓

Complexity: naive vs. optimal

Approach	Time	At $n = 500$
Full game tree (naive)	$O(2^n)$	$\approx 3 \times 10^{150}$
Interval DP (score difference)	$O(n^2)$	250,000

The naive recursion is not merely slower – it visits 2^{500} tree nodes to answer a question that only has $\binom{501}{2} \approx 125,250$ *distinct* states. Recognizing “how many distinct states, really?” is the entire gap.

Python implementation

```
1 def stoneGame(piles):
2     n = len(piles)
3     dp = [[0] * n for _ in range(n)]
4     for i in range(n):
5         dp[i][i] = piles[i]                # window of length 1
6
7     for length in range(2, n + 1):         # grow the window
8         for i in range(n - length + 1):
9             j = i + length - 1
10            dp[i][j] = max(piles[i] - dp[i + 1][j],
11                           piles[j] - dp[i][j - 1])
12
13     return dp[0][n - 1] > 0
```

Filling by increasing length guarantees $dp[i+1][j]$ and $dp[i][j-1]$ (both *shorter* windows) are already computed before $dp[i][j]$ needs them.

Implementation tips & tricks (the programming side)

- **Fill order is not optional.** The recurrence needs shorter windows computed first; a loop nested the “obvious” way (outer i , inner j) accidentally reads $dp[i][j - 1]$ or $dp[i + 1][j]$ *before* it has been written. Looping by `length` (as above) sidesteps the bug entirely instead of reasoning about loop-order edge cases.
- `@lru_cache` **recursion vs. the bottom-up array.** A memoized recursive version is easier to write correctly first (translate the recurrence almost verbatim) but pays Python function-call overhead per state and needs `sys.setrecursionlimit` raised for $n = 500$; the bottom-up array above has neither cost and is the version to submit.
- **Space can drop to $O(n)$.** Each diagonal only reads the *previous* diagonal's values, so a single rolling 1-D array (updated carefully in decreasing i order within each `length` pass) suffices – a standard interval-DP space optimization once the $O(n^2)$ version is verified correct.

Problem statement

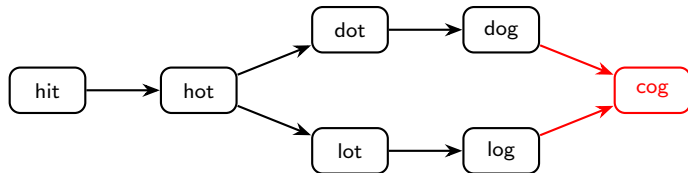
LeetCode 127 – Word Ladder (**Hard**)

Given `beginWord`, `endWord`, and a dictionary `wordList`, return the number of words in the **shortest** transformation sequence from `beginWord` to `endWord`, where each step changes exactly **one letter** and every intermediate word must appear in `wordList`. Return 0 if no such sequence exists.

- Constraints (LeetCode): $1 \leq \text{wordList.length} \leq 5000$, all words the same length $L \leq 10$.
- Reframe it as a graph problem: nodes are words, an edge joins two words that differ in exactly one letter, and the answer is a **shortest path** in this (huge, implicit) graph – exactly this week's “exponential-looking search space, tamed by exploring it the right way” theme.

Worked example

```
beginWord = "hit", endWord = "cog"  
wordList = ["hot", "dot", "dog", "lot", "log", "cog"]
```



Two equally-short paths exist: hit-hot-dot-dog-cog and hit-hot-lot-log-cog, both length 5.

Answer: 5.

First instinct: search every path

Most direct reading of the problem

“Find the shortest transformation sequence” $\Rightarrow$ DFS/backtrack from beginWord, trying every one-letter change at each step, exploring every path to endWord, and keep the shortest one found.

```
1: function DFS(word, path)
2:   if word = endWord then record len(path); return
3:   end if
4:   for each 1-letter variant  $v$  of word in wordList do
5:     DFS( $v$ , path + [ $v$ ])            $\triangleright$  no record of what's already been tried!
6:   end for
7: end function
```

Why this is bad

- Without tracking visited words, the search can loop between words forever (`hot` $\rightarrow$ `dot` $\rightarrow$ `hot` $\rightarrow$ $\dots$) – it may not even **terminate**.
- Even with a visited check per path (not globally), DFS must explore *every* path before it can be sure it found the *shortest* one – in a graph with up to 5000 words and roughly $25L \approx 250$ possible one-letter edits per word, the number of distinct paths explodes combinatorially long before reaching `endWord`.

The smell to notice

“Explore every path to find the *shortest* one” is always the wrong move in an unweighted graph – **BFS** finds shortest paths by construction, without ever exploring a path longer than necessary.

Structural insight #1: this is unweighted shortest path

- Every edge (one-letter change) has the same “cost” – so the shortest transformation sequence is exactly the **BFS shortest path** from beginWord to endWord, layer by layer, with a **global visited set** so no word is ever processed twice.
- This bounds the work to (number of words) $\times$ (edits per word) instead of (number of *paths*) – BFS visits each of the ≤ 5000 words **at most once**.

Structural insight #2: how you *find* neighbors matters too

- **Tier 1 (correct, but not optimal):** precompute the graph by comparing *every pair* of words for a one-letter difference: $O(N^2L)$ – at $N=5000$, $L=10$, that is 2.5×10^8 character comparisons.
- **Tier 2 (optimal):** from each word, *generate* its $\leq 25L$ one-letter variants directly and test dictionary membership in $O(1)$ via a hash set – $O(NL \cdot 26)$ total, about 1.3×10^6 operations at the same constraints, roughly $190\times$ fewer.
- The lesson: even after picking the right *algorithm* (BFS), *how* you enumerate a node's neighbors can itself be an asymptotic improvement.

BFS trace on the worked example

Visited set starts as {hit}; wordSet = {hot,dot,dog,lot,log,cog}.

layer (length)	frontier	how it was reached
1	hit	start
2	hot	hit→hot (1 letter); dot, lot, log, dog, cog each differ from hit by ≥ 2 letters – not neighbors
3	dot, lot	both differ from hot by exactly 1 letter; log, dog, cog do not
4	dog, log	dog from dot; log from lot (each 1 letter)
5	cog	cog reachable from <i>both</i> dog and log – first one processed wins, length is the same either way

cog first appears at layer 5 $\Rightarrow$ **answer: 5** – matches the expected output.

Optimal algorithm: pseudocode

```
1: function WORDLADDER(beginWord, endWord, wordList)
2:   words  $\leftarrow$  set(wordList)
3:   if endWord  $\notin$  words then return 0
4:   end if
5:   queue  $\leftarrow$  [(beginWord, 1)]; remove beginWord from words
6:   while queue not empty do
7:     (word, dist)  $\leftarrow$  queue.popleft()
8:     if word = endWord then return dist
9:     end if
10:    for each position p in word, each letter c  $\neq$  word[p] do
11:      v  $\leftarrow$  word with position p replaced by c
12:      if v  $\in$  words then
13:        remove v from words; enqueue (v, dist + 1)
14:      end if
15:    end for
16:  end while
17:  return 0
18: end function
```

▷ $O(1)$ membership

$O(NL \cdot 26)$ total – each of the $\leq N$ words is enqueued once.

Complexity: naive vs. optimal

Approach	Time	At $N = 5000$, $L = 10$
DFS, explore every path	exponential	does not finish
BFS over precomputed all-pairs graph	$O(N^2L)$	$\approx 2.5 \times 10^8$
BFS, generate variants on the fly	$O(NL \cdot 26)$	$\approx 1.3 \times 10^6$

Two separate wins stack: BFS over DFS (bounds work by *states*, not *paths*), and generate-neighbors over compare-all-pairs (bounds work by *edits*, not *other words*).

Python implementation (part 1: setup)

```
1  from collections import deque
2  import string
3
4  def ladderLength(beginWord, endWord, wordList):
5      words = set(wordList)
6      if endWord not in words:
7          return 0
8
9      queue = deque([(beginWord, 1)])
10     words.discard(beginWord)      # double duty: dictionary AND visited set
```


Python implementation (part 2: BFS loop)

```
1  while queue:
2      word, dist = queue.popleft()
3      if word == endWord:
4          return dist
5      for i in range(len(word)):
6          prefix, suffix = word[:i], word[i + 1:]
7          for c in string.ascii_lowercase:
8              if c == word[i]:
9                  continue
10             candidate = prefix + c + suffix
11             if candidate in words:
12                 words.discard(candidate)    # mark visited before enqueueing
13                 queue.append((candidate, dist + 1))
14
15  return 0
```

Implementation tips & tricks (the programming side)

- `deque.popleft()`, **never** `list.pop(0)`. Popping the front of a Python list is $O(n)$ (it shifts every remaining element); `collections.deque` makes it $O(1)$. Using a plain list here would silently turn an $O(NL \cdot 26)$ algorithm back into something quadratic in the queue size.
- **Reuse the dictionary set as the visited set.** Calling `words.discard(candidate)` the moment a word is *enqueued* (not when it is *processed*) means membership testing and visited-tracking are the same $O(1)$ hash-set operation – no second data structure needed, and no risk of the same word being enqueued twice from two different parents in the same layer.
- **Build each candidate with slicing, not a mutable list.** `prefix + c + suffix` on Python strings is a small, cheap allocation; a tempting alternative (convert to `list(word)`, mutate one index, `"".join(...)`) does strictly more work for the same result.
- **Bonus (borderline algorithmic):** *bidirectional* BFS – growing frontiers from `beginWord` and `endWord` simultaneously, always expanding whichever frontier is smaller – is the standard further speed-up for this exact problem, cutting the effective branching from b^d to roughly $2b^{d/2}$.

Summary: two problems, one lesson

- 1 **Stone Game** has an exponential-looking game tree (2^n move sequences), but only $O(n^2)$ *genuinely distinct* states (contiguous windows) – memoized interval DP collapses the tree onto those states.
- 2 **Word Ladder** has an exponential-looking search space (every path through an implicit word graph), but BFS bounds the work by the number of *states* (words) rather than the number of *paths* between them – and generating neighbors on the fly (instead of precomputing all pairs) shaves a further factor of $N/26L$ off that.
- 3 Both times, the naive instinct was “enumerate everything that could happen”; both times, the fix was to notice that *most of what could happen revisits the same underlying state*, and to compute each state’s answer exactly once – the same move as this week’s bounded search trees and memoized game DP.
- 4 The implementation tricks (fill order, deque vs. list, reusing one set for two jobs) are a second, independent layer: even the *optimal* algorithm can regress to a worse complexity class through an innocent-looking data-structure choice.

Keep practicing

LEETCODE.md lists six more Week 13 problems (*Sliding Puzzle*, *Smallest Sufficient Team*, *Can I Win*, *Cat and Mouse*, ...) – try applying the same four-step process to each of them.